

Unit 1

Introduction of C Language

Detailed Study Notes — Fundamentals of Programming in C

Topics Covered: Computer Languages, Programming Concepts, History of C, Traditional vs Modern C, Character Set, Tokens, Operators, Operator Precedence, Type Casting, Data Types, and Preprocessors.

1. Introduction of Computer Languages

A computer language (or programming language) is a formal set of instructions, rules, and syntax used to communicate with a computer and direct it to perform specific tasks. Since a computer only understands electrical signals in the form of binary digits, every program, regardless of the language it is written in, must eventually be translated into a sequence of 0s and 1s before it can be executed. Computer languages are broadly categorized into three levels based on how close they are to human language versus machine hardware.

1.1 Machine Language (Low-Level)

Machine language is the most fundamental and lowest level of programming language. It consists entirely of binary digits — 0s and 1s — that represent electronic signals (on/off states) understood directly by the computer's central processing unit (CPU).

- **Directly executable:** No translator or compiler is needed since the CPU understands it natively.
- **Hardware-dependent:** Machine code written for one processor architecture (e.g., Intel x86) will not run on a different architecture (e.g., ARM) without modification.
- **Extremely difficult for humans:** Writing or debugging binary instructions is tedious, error-prone, and time-consuming.
- **Fastest execution:** Since no translation step is required at run time, machine code executes the fastest.

Example (conceptual): An instruction such as 1010 1101 0001 1110 might instruct the CPU to add the contents of two specific registers.

1.2 Assembly Language (Low-Level)

Assembly language improves upon machine language by replacing binary sequences with short, human-readable abbreviations called mnemonics (e.g., ADD, SUB, MOV, JMP). Each assembly instruction typically corresponds to exactly one machine language instruction.

- Requires an **assembler**, a special program that converts assembly mnemonics into machine code.
- Still **architecture-specific** — assembly code for one processor family generally cannot run on another.
- Easier to read and write than pure binary, but still requires deep knowledge of the underlying hardware, registers, and memory addressing.
- Commonly used today in device drivers, embedded systems, operating system kernels, and performance-critical routines.

Example: MOV AL, 5 ; move the value 5 into register AL
ADD AL, BL ; add the value in register BL to AL

1.3 High-Level Language

High-level languages are designed to be easy for humans to read, write, and understand. Their syntax resembles natural (English-like) language and mathematical notation rather than hardware-specific instructions. Examples include C, C++, Java, Python, and JavaScript.

- **Machine-independent (portable):** The same source code can often run on different types of computers with little or no change.
- **Requires a compiler or interpreter** to translate the high-level instructions into machine language.
- **Easier to learn, write, and debug** compared to low-level languages, increasing programmer productivity.
- **Rich feature set:** Supports variables, control structures, functions, data structures, and libraries that simplify complex programming tasks.

1.4 Translators: Compiler vs Interpreter vs Assembler

Translator	Works On	How It Works
Assembler	Assembly Language	Converts assembly mnemonics directly into machine code, one instruction at a time.
Compiler	High-Level Language	Translates the entire source code into machine code (object code) at once, before execution. Errors are reported together after the whole file is scanned.
Interpreter	High-Level Language	Translates and executes the program line-by-line. Execution stops as soon as an error is encountered.

C uses a **compiler** to convert source code into an executable file, which is why C programs generally run faster than interpreted languages.

2. Introduction of Programming Concepts

Programming is the process of designing, writing, testing, and maintaining the source code of computer programs to accomplish a specific computing task. Before writing actual code, a programmer typically plans the logic of the solution using algorithms and flowcharts.

2.1 Algorithm

An algorithm is a finite, well-defined, step-by-step logical procedure used to solve a specific problem or perform a computation. A good algorithm should be unambiguous, have a clear starting and ending point, and produce the correct output for valid input in a finite number of steps.

Example — Algorithm to find the sum of two numbers:

```
Step 1: Start
Step 2: Read two numbers, A and B
Step 3: Compute Sum = A + B
Step 4: Display Sum
Step 5: Stop
```

2.2 Flowchart

A flowchart is a diagrammatic representation of an algorithm, using standardized geometric symbols connected by arrows to show the flow of control and logic through a process.

Symbol	Shape	Meaning
Terminator	Oval / Rounded Rectangle	Represents the Start or End (Stop) of the program.
Input/Output	Parallelogram	Represents reading input data or displaying output.
Process	Rectangle	Represents a processing step, such as a calculation or assignment.
Decision	Diamond	Represents a decision point with two or more possible outcomes (Yes/No, True/False).
Connector	Small Circle	Used to connect different parts of a flowchart, especially across pages.
Flow Line	Arrow	Indicates the direction/sequence of execution.

2.3 Source Code, Object Code, and Executable Code

- **Source Code:** The human-readable program written by the programmer in a high-level language (saved with extension `.c` for C programs).
- **Object Code:** The intermediate machine-code translation of the source file generated by the compiler (extension `.obj` or `.o`). It is not directly runnable on its own.
- **Executable Code:** The final linked binary file (e.g., `.exe`) produced after the linker combines object code with required libraries; this file can be run directly by the operating system.

2.4 Compilation Process

Compilation is the process of translating high-level source code into machine-executable code. In C, this happens in four broad stages:

- **Pre-processing:** The preprocessor handles directives (lines starting with `#`), such as file inclusion and macro expansion, producing an expanded source file.

- **Compilation:** The compiler translates the pre-processed code into assembly language, checking for syntax errors.
- **Assembly:** An assembler converts the assembly code into object code (machine code) for the target architecture.
- **Linking:** The linker combines the object code with required library functions (like those from `stdio.h`) to produce the final executable file.

3. Introduction of C Language (History & Overview)

3.1 History

C is a general-purpose, procedural, structured programming language. It was developed in 1972 by Dennis Ritchie at Bell Telephone Laboratories (part of AT&T) as an evolution of the earlier 'B' language (created by Ken Thompson), which itself was derived from BCPL (Basic Combined Programming Language). The primary motivation for creating C was to develop the UNIX operating system, which had originally been written in assembly language and needed a more portable, maintainable alternative.

By 1973, most of the UNIX kernel had been rewritten in C — a landmark event that proved a systems-level operating system could be written in a high-level language without sacrificing much performance. C's popularity subsequently spread rapidly beyond Bell Labs during the late 1970s and 1980s, and it became the foundation language for countless later languages, including C++, Java, C#, and even parts of Python's own interpreter.

In 1983, the American National Standards Institute (ANSI) formed a committee to establish a standard specification for C, resulting in **ANSI C** (also called **C89**) in 1989. This was later adopted internationally as **ISO C**. Subsequent revisions include C99, C11, C17, and C23, each adding new features while retaining backward compatibility with the core language.

3.2 Overview and Key Characteristics

- **Mid-level Language:** C combines the efficiency and hardware-access capability of low-level languages (like pointers and direct memory manipulation) with the readability and structure of high-level languages. This makes it suitable for both system software (operating systems, drivers) and application software.
- **Structured / Procedural Language:** A C program is organized into smaller, reusable blocks called functions, each performing a specific task. This promotes modularity, code reuse, and easier debugging.
- **Highly Portable:** A C program written and compiled on one machine/operating system can typically be compiled and run on another system with little or no modification, as long as a standard-compliant compiler is available.
- **Rich Set of Operators:** C provides a wide variety of operators, including bitwise operators, for fine-grained control over data.
- **Fast Execution:** Because it is compiled directly to machine code and gives programmers close control over memory, C programs run very efficiently compared to many interpreted languages.
- **Extensible:** New functionality can easily be added through user-defined functions and libraries.
- **Foundation for Modern Languages:** Many modern languages, such as C++, Java, and C#, derive their basic syntax (loops, conditionals, operators, block structure) from C.

Applications of C: Operating systems (large parts of UNIX, Linux, and Windows components), embedded systems and firmware, database engines (e.g., core components of MySQL, PostgreSQL), compilers and interpreters for other languages, and performance-critical software.

4. Difference between Traditional and Modern C

'Traditional C' generally refers to the original K&R C, described in the book '*The C Programming Language*' by Brian Kernighan and Dennis Ritchie (1978), before any official standardization. 'Modern C' refers to the standardized versions beginning with ANSI C (C89) and continuing through C99, C11, C17, and beyond.

Feature	Traditional C (K&R C — 1978)	Modern C (ANSI C / C89 / C99 / C11)
Function Declaration	Parameter types were declared separately, below the function header line.	Introduced function prototypes, where parameter types are declared inside the parentheses of the function signature itself, enabling stricter compile-time type checking.
Standard Library	Very limited, inconsistent standard libraries across implementations.	Rich, standardized set of libraries such as <stdio.h>, <stdlib.h>, <string.h>, <math.h>, and <time.h>.
Comments	Only block comments /* ... */ were supported.	Introduced single-line comments // (officially from C99 onward), in addition to block comments.
Variable Declaration	All variables had to be declared at the very beginning of a block, before any executable statement.	Variables can be declared anywhere in the code, as long as they appear before their first use (from C99 onward).
Type Checking	Weak / minimal type checking; many type mismatches went undetected at compile time.	Stronger type checking through prototypes and standardized type rules, catching more errors at compile time.
Data Types	Fewer built-in types; no boolean type.	Additional types and keywords such as _Bool (C99), long long, and improved support for complex numbers.
Portability	Behavior could vary significantly between different compilers/vendors.	A formal standard ensures much more consistent behavior across conforming compilers.

4.1 Example: Function Declaration Style

Traditional (K&R) style:

```
int add(a, b)
int a, b;
{
    return a + b;
}
```

Modern (ANSI) style:

```
int add(int a, int b)
{
    return a + b;
}
```

5. C Character Set

The C character set refers to the complete collection of valid characters that a C compiler recognizes and can use to build a program. Any character outside this set that appears in the wrong context will typically be flagged as a syntax error by the compiler (though such characters may still appear legally inside string literals, character constants, or comments).

5.1 Components of the Character Set

- **Alphabets:** Uppercase letters A–Z and lowercase letters a–z (52 letters total). C is case-sensitive, so 'A' and 'a' are treated as different characters.
- **Digits:** The ten decimal digits 0 through 9.
- **Special Characters:** Symbols used for operators, punctuation, and syntax, e.g. ! * + \ " < > # % () { } [] ; : , . / ? & ^ - _ = | ~ and the single quote character.
- **White Spaces:** Blank space, horizontal tab (\t), carriage return (\r), new line (\n), and form feed. White space is generally used to separate tokens and improve readability; it is otherwise ignored by the compiler.

In addition to this basic set, the C99 and later standards also support an extended character set through **trigraphs** and **escape sequences** (such as \n for newline, \t for tab, \\ for a literal backslash, and \" for a literal double quote) that allow representation of characters that would otherwise be difficult to type or that have special meaning.

6. C Tokens

A token is the smallest individual unit in a C program that is meaningful to the compiler. During compilation, the compiler's lexical analyzer scans the source code from left to right and breaks it down into a stream of tokens. C recognizes six major categories of tokens: keywords, identifiers, constants, strings, operators, and special symbols (punctuators).

A. Keywords

Keywords are reserved words that carry special, predefined meaning to the C compiler. Because their meaning is fixed by the language, keywords cannot be used as identifiers (i.e., they cannot be used as names for variables, functions, or arrays). The ANSI C standard defines **32 standard keywords**. Later standards (C99, C11) added a few more, often prefixed with an underscore, such as `_Bool` and `_Complex`.

The 32 ANSI C keywords:

```
auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while
```

B. Constants

Constants are fixed literal values that do not change at any point during the execution of a program. C supports several kinds of constants:

- **Integer Constants:** Whole numbers without a decimal point, e.g. 10, -50, 0. These can also be written in octal (prefix 0, e.g. 017) or hexadecimal (prefix 0x, e.g. 0x1F).
- **Real / Floating-point Constants:** Numbers containing a decimal point or exponent, e.g. 3.14, -0.5, 2.5e3.
- **Character Constants:** A single character enclosed in single quotes, e.g. 'A', '5', '\n'. Internally stored as the character's corresponding integer (ASCII) value.
- **String Constants:** A sequence of characters enclosed in double quotes, e.g. "Hello". Every string constant is automatically terminated by a null character.

C. Strings

A string in C is technically not a distinct built-in data type but rather an array of characters (`char`) terminated by a special null character `'\0'` (ASCII value 0). This null terminator marks the end of the string and is essential — string-handling functions like `strlen()` and `printf()` rely on it to know where the string ends.

```
char greeting[] = "Hello";  
// Stored in memory as: 'H' 'e' 'l' 'l' 'o' '\0'  
// The compiler automatically appends '\0' at the end.
```

D. Identifiers and Variables

Identifiers are the names given to various program elements such as variables, functions, arrays, and structures, so that they can be referred to elsewhere in the program.

Rules for naming identifiers:

- Must begin with a letter (A–Z, a–z) or an underscore (`_`); it cannot begin with a digit.
- After the first character, it may contain letters, digits, and underscores — but no other special characters or spaces.
- Cannot be a reserved keyword (e.g., you cannot name a variable `'int'` or `'while'`).
- C is case-sensitive, so identifiers `'Total'` and `'total'` are treated as two completely different names.

- There is generally no limit on length, though only the first 31 (or 63, depending on the standard/compiler) significant characters may be guaranteed to be distinguished by some older compilers.

Variables are named memory locations used to store data values that can be read and modified while the program is running.

```
int age = 25;          // 'age' is an identifier/variable storing the integer value 25
float price = 99.5;    // 'price' stores a floating-point value
char grade = 'A';      // 'grade' stores a single character
```

E. Operators (All 8 Types)

Operators are special symbols that instruct the compiler to perform specific mathematical, relational, logical, or bitwise operations on operands (variables and/or constants). C provides a very rich set of operators, broadly grouped into eight categories.

i. Arithmetic Operators

Used to perform basic mathematical calculations on numeric operands.

Operator	Meaning	Example (a=10, b=3)
+	Addition	a + b = 13
-	Subtraction	a - b = 7
*	Multiplication	a * b = 30
/	Division	a / b = 3 (integer division)
%	Modulo (remainder)	a % b = 1

ii. Relational Operators

Used to compare two operands; the result of a relational expression is always either 1 (true) or 0 (false).

- == Equal to
- != Not equal to
- > Greater than
- < Less than
- >= Greater than or equal to
- <= Less than or equal to

iii. Logical Operators

Used to combine or invert conditional (relational) expressions; commonly used in decision-making statements (if, while).

- && Logical AND — true only if both operands are true.
- || Logical OR — true if at least one operand is true.
- ! Logical NOT — reverses the logical state of its operand.

iv. Assignment Operators

Used to assign a value to a variable. C also provides compound (shorthand) assignment operators that combine an arithmetic operation with assignment.

```
a = 5;      // simple assignment
a += 5;     // equivalent to a = a + 5
a -= 3;     // equivalent to a = a - 3
a *= 2;     // equivalent to a = a * 2
a /= 4;     // equivalent to a = a / 4
a %= 3;     // equivalent to a = a % 3
```

v. Increment / Decrement Operators

Unary operators that increase or decrease the value of a variable by exactly 1.

- ++ Increment — increases the value by 1.
- -- Decrement — decreases the value by 1.

- **Prefix form** (++a): the variable is updated *before* its value is used in the expression.
- **Postfix form** (a++): the current value is used in the expression *first*, and the variable is updated afterward.

```
int a = 5;
int x = ++a; // a becomes 6, then x = 6 (pre-increment)
int b = 5;
int y = b++; // y = 5 first, then b becomes 6 (post-increment)
```

vi. Conditional (Ternary) Operator

The only operator in C that takes three operands, providing a compact shorthand for a simple if-else structure.

```
condition ? Expression1 : Expression2;
// If 'condition' is true (non-zero), Expression1 is evaluated and returned.
// Otherwise, Expression2 is evaluated and returned.

int a = 10, b = 20, max;
max = (a > b) ? a : b; // max = 20
```

vii. Bitwise Operators

Operate directly on the individual bits of integer operands. Commonly used in low-level programming, embedded systems, and performance optimizations.

Operator	Name	Description
&	Bitwise AND	Sets each bit to 1 only if both corresponding bits are 1.
	Bitwise OR	Sets each bit to 1 if at least one of the corresponding bits is 1.
^	Bitwise XOR	Sets each bit to 1 only if the corresponding bits are different.
<<	Left Shift	Shifts bits to the left, filling with 0s (equivalent to multiplying by powers of 2).
>>	Right Shift	Shifts bits to the right (equivalent to dividing by powers of 2 for unsigned values).
~	One's Complement	Inverts every bit of the operand (1 becomes 0, 0 becomes 1).

viii. Special Operators

- `sizeof()` — Returns the size (in bytes) occupied by a variable or data type.
- `&` — The address-of operator; returns the memory address of a variable.
- `*` — The pointer/dereference operator; declares a pointer, or accesses the value stored at an address.
- `,` — The comma operator: used to link/separate related expressions. evaluated left to right.

```
int x = 10;
printf("%lu", sizeof(x)); // prints the size of int, typically 4
int *p = &x;              // p now stores the address of x
printf("%d", *p);         // *p dereferences p, printing 10
```

7. Hierarchy of Operators (Operator Precedence)

When an expression contains more than one operator, **precedence** determines which operator is evaluated first. **Associativity** determines the order of evaluation (left-to-right or right-to-left) when two or more operators of the *same* precedence level appear together in an expression.

Precedence	Operators	Associativity
1. Highest	() [] -> .	Left to Right
2. Unary	++ -- ! sizeof (type)	Right to Left
3. Multiplicative	* / %	Left to Right
4. Additive	+ -	Left to Right
5. Shift	<< >>	Left to Right
6. Relational	< <= > >=	Left to Right
7. Equality	== !=	Left to Right
8. Bitwise AND	&	Left to Right
9. Bitwise XOR	^	Left to Right
10. Bitwise OR		Left to Right
11. Logical AND	&&	Left to Right
12. Logical OR		Left to Right
13. Conditional	? :	Right to Left
14. Assignment (Lowest)	= += -= *= /= %=	Right to Left
15. Comma	,	Left to Right

Worked Example:

```
int result = 10 + 5 * 2;
// '*' has higher precedence than '+', so it is evaluated first:
// result = 10 + (5 * 2) = 10 + 10 = 20
```

Parentheses can always be used to explicitly override the default precedence and force a particular order of evaluation, which also improves code readability.

8. Type Casting

Type casting is the process of converting a variable of one data type into another data type. This is necessary because C is a statically typed language, and certain operations (such as mixed-type arithmetic or function calls expecting a specific type) require operands to be of a compatible or matching type.

8.1 Implicit Type Casting (Automatic / Type Promotion)

Implicit conversion is performed automatically by the compiler, without any explicit instruction from the programmer, typically when a 'smaller' or lower-ranked data type is combined with a 'larger' or higher-ranked one in an expression. This follows C's usual arithmetic conversion rules — for example, an `int` is automatically promoted to `float` when used together in an arithmetic expression, to avoid loss of information.

```
int a = 5;
float b = 2.5;
float sum = a + b;    // 'a' is implicitly converted to float before addition
// sum = 7.5
```

8.2 Explicit Type Casting

Explicit conversion is performed manually by the programmer using the cast operator, written as the target type in parentheses placed before the value or variable: `(type) expression`. This gives the programmer direct control over how and when a conversion happens.

```
int a = 5, b = 2;
// Without casting, 5 / 2 results in 2 (integer division discards the remainder).
int result1 = a / b;           // result1 = 2

// With explicit casting, one operand is forced to float before division:
float result2 = (float)a / b;  // result2 = 2.5
```

Common use-cases for explicit casting:

- Forcing floating-point (real) division instead of truncating integer division.
- Converting a double to an `int` to intentionally discard the fractional part.
- Converting between pointer types when working with generic or low-level memory operations.
- Ensuring a function receives an argument of the exact type it expects.

Caution: Explicit casting can lead to data loss if converting from a larger type to a smaller one (e.g., double to `int`, or `int` to `char`), since some information may be truncated or overflow may occur.

9. Data Types in C

A data type specifies the kind of value a variable can hold, how much memory it occupies, and what range of values and operations are valid for it. C's data types are broadly classified into three groups: primary (built-in), derived, and user-defined.

9.1 Primary (Built-in) Data Types

Data Type	Description	Typical Size	Format Specifier
int	Stores whole numbers (positive, negative, or zero), no decimal part.	2 or 4 bytes	%d
char	Stores a single character or a small integer (its ASCII code).	1 byte	%c
float	Stores single-precision decimal (floating-point) numbers.	4 bytes	%f
double	Stores double-precision decimal numbers, offering greater range and accuracy than float.	8 bytes	%lf
void	Represents the absence of a type; used for functions that return no value, or generic pointers.	—	—

C also provides **type qualifiers/modifiers** — `signed`, `unsigned`, `short`, and `long` — which can be combined with `int` and `double` to adjust the range and size of values that can be stored (e.g., `unsigned int`, `long double`, `short int`).

9.2 Derived Data Types

Derived types are built from the primary data types and allow more complex data organization.

- **Arrays:** A fixed-size, ordered collection of elements of the same data type, stored in contiguous memory locations (e.g., `int marks[10];`).
- **Pointers:** Variables that store the memory address of another variable, rather than a direct value (e.g., `int *ptr;`).
- **Functions:** Named, reusable blocks of code designed to perform a specific task, which can accept inputs (parameters) and return a value.

9.3 User-Defined Data Types

These allow programmers to create their own custom, more meaningful data types by combining existing ones.

- **struct (Structure):** Groups together variables of different data types under a single name, allowing related data to be handled as one unit (e.g., a 'Student' structure holding a name, roll number, and marks).
- **union:** Similar to a structure, but all its members share the same memory location, so a union only occupies enough memory for its largest member at any given time.
- **enum (Enumeration):** Assigns descriptive names to a set of integer constants, improving code readability (e.g., `enum Day {MON, TUE, WED, ...}`).

```
struct Student {
    char name[50];
    int rollNo;
    float marks;
```

```
};
```

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }; // MON=0, TUE=1, ... SUN=6
```

10. Pre-processors in C

The C Preprocessor (often abbreviated CPP) is a macro-processing tool that runs automatically before the actual compilation of the source code begins. It scans the source file for special instructions called **preprocessor directives**, all of which begin with the hash/pound symbol #, and transforms the source code accordingly (for example, by inserting file contents or replacing macro names with their defined values) before handing the expanded code to the compiler proper.

10.1 #include Directive

Used to insert the entire contents of another file (typically a header file) into the source file at the point of inclusion. This is how a program gains access to standard library functions such as `printf()` and `scanf()`.

- `#include <stdio.h>` — Angle brackets tell the preprocessor to search for the file in the compiler's standard system include directories. Used for standard library headers.
- `#include "myheader.h"` — Double quotes tell the preprocessor to search first in the current directory (the programmer's own project files) before falling back to the system directories. Used for user-defined header files.

10.2 #define Directive

Used to define a symbolic constant or a macro. Wherever the defined name subsequently appears in the code, the preprocessor textually substitutes it with its defined value before compilation begins. This is often used to make code more readable and easier to maintain (changing a value in one place updates every use throughout the program).

```
#include <stdio.h>
#define PI 3.14159          // Object-like macro (symbolic constant)
#define SQUARE(x) ((x) * (x)) // Function-like macro

int main() {
    float radius = 5.0;
    float area = PI * radius * radius;    // PI is replaced with 3.14159 here
    printf("Area is: %f\n", area);
    printf("Square of 4 is: %d\n", SQUARE(4)); // Expands to ((4) * (4))
    return 0;
}
```

10.3 Other Common Preprocessor Directives

Directive	Purpose
<code>#undef</code>	Removes (undefines) a previously defined macro.
<code>#ifdef</code> / <code>#ifndef</code>	Conditionally compiles a block of code only if a macro is (or is not) already defined.
<code>#if</code> / <code>#else</code> / <code>#elif</code> / <code>#endif</code>	Conditional compilation based on the evaluation of a constant expression.
<code>#pragma</code>	Gives special, compiler-specific instructions (e.g., suppressing certain warnings).
<code>#error</code>	Forces the compiler to stop and display a custom error message.

A very common and important use of conditional directives is the **include guard**, which prevents a header file from being included (and thus its contents processed) more than once within the same compilation, avoiding duplicate-definition errors.

```
#ifndef MYHEADER_H
#define MYHEADER_H

// ... header file contents ...

#endif
```

End of Unit 1 — Introduction of C Language

Unit

Introduction of Logic Development Tools

Detailed Study Notes — Programming Logic, Flowcharts, Dry-Runs, and Design Techniques

Topics Covered: Introduction of Logic, Steps for Developing Logic, Basics of Flowcharts and their Symbols, Dry-Run / Desk-Checking with Trace Tables, and other Logic Development Techniques including Pseudocode, Structure Charts, and Decision Tables.

1. Introduction of Logic

In computer science, **logic** refers to the foundational thought process and sequential reasoning used to analyze and solve a problem. Before writing a single line of code in C, or indeed in any programming language, a programmer must first work out precisely what steps the computer needs to follow, in what order, to transform the given inputs into the desired output. Writing code without first developing sound logic almost always leads to programs that are buggy, inefficient, or simply incorrect.

1.1 Logic is Language-Independent

One of the most important ideas to internalize is that logic itself is universal and does not belong to any single programming language. The underlying logic required to, say, sort a list of numbers or search for an item in an array is exactly the same whether it is eventually implemented in C, Python, Java, or any other language — only the specific syntax used to express that logic changes from one language to another. This is why strong logical thinking, once developed, transfers easily across different programming languages and technologies.

1.2 Core Components of Programming Logic

Virtually all programming logic, no matter how complex, can be broken down into combinations of just three fundamental building blocks, often referred to as the three pillars of structured programming:

- **Sequence:** Executing a series of instructions one after another, in a fixed, straightforward order — the default flow of any program.
- **Selection:** Making a decision between two or more possible paths of execution, based on whether a condition is true or false, using constructs such as if/else and switch.
- **Iteration (Repetition):** Repeating a set of instructions multiple times, either a fixed number of times or until some condition is met, using loop constructs such as while, do-while, and for.

This principle — that any well-structured algorithm can be built purely from sequence, selection, and iteration — is formally known as the **Structured Program Theorem**, and it underlies the design of virtually every modern programming language, including C.

2. Necessary Instructions for Developing Logic

Developing strong, reliable programming logic requires a systematic and disciplined approach rather than jumping straight into writing code. Experienced programmers typically follow a structured sequence of steps before they type out their first line of C:

Step 1: Understand the Problem (Requirements Gathering)

The very first step is to clearly and precisely define exactly what the program needs to achieve. This means identifying the exact inputs the program will receive, the exact output it must produce, and any constraints or rules that govern the relationship between them. A surprisingly large fraction of programming errors trace back to a misunderstanding of the problem itself, rather than a mistake in the code.

Step 2: Decomposition (Break It Down)

Large, complex problems can be overwhelming if tackled all at once. Decomposition means breaking the main problem down into smaller, self-contained, and more manageable sub-problems or modules — each of which is much easier to design, code, test, and debug in isolation. In C, this naturally maps onto writing separate functions for each sub-problem.

Step 3: Identify Variables and States

Next, determine exactly what pieces of data need to be stored in memory while the program runs — for example, counters that track how many times something has happened, running totals or sums, flags that record whether a certain event has occurred, and temporary values used during calculations. Identifying these early makes it much easier to design the algorithm around them.

Step 4: Formulate the Algorithm

With the problem understood, broken down, and the necessary variables identified, the programmer writes out the step-by-step instructions — the algorithm — that will transform the given inputs into the required outputs. This is typically expressed in plain language or pseudocode at this stage, deliberately kept independent of any specific programming language's syntax.

Step 5: Anticipate Edge Cases

Finally, a careful programmer thinks ahead about unusual, extreme, or unexpected inputs that the program might encounter — for example: What happens if the user enters a negative number when a positive one was expected? What if they enter zero? What if a list that is supposed to be searched is actually empty? What if two values that are supposed to be different turn out to be equal? Anticipating and explicitly handling these edge cases up front, rather than after a program has already failed, is one of the clearest marks of a mature and disciplined programmer.

Tip: A useful habit is to write down a short list of test inputs — including at least one 'normal' case and two or three 'edge' cases — before writing the algorithm itself. This makes it much easier to check the logic later during a dry-run.

3. Basics of Flow Chart

A flowchart is a graphical, diagrammatic representation of an algorithm or a logical sequence of steps. It uses a set of standardized geometric symbols, connected by arrows, to visually represent the different types of instructions and the order in which they occur. Because a flowchart communicates logic visually rather than through text, it is often much easier to understand at a glance, to review with others, and to spot logical gaps or errors in, compared to a purely written description of the same algorithm.

3.1 Standard Flowchart Symbols

Symbol Shape	Name	Purpose
Oval / Pill	Start / Stop (Terminator)	Indicates the beginning or the end of the program or process. Every flowchart must have exactly one Start and at least one Stop symbol.
Parallelogram	Input / Output	Represents an operation that reads data into the program (input) or displays/prints data from the program (output).
Rectangle	Process	Represents a processing step, such as a calculation, an assignment, or any other data manipulation that does not involve a decision or I/O.
Diamond	Decision	Represents a condition or question (e.g., "Is $X > Y$?"). It always has two or more output paths, typically labeled True/Yes and False/No, leading to different parts of the flowchart.
Arrows	Flow Lines	Connects the various symbols together and indicates the direction in which execution proceeds through the flowchart.
Small Circle	Connector	Used to join different parts of a flowchart, especially when the diagram needs to continue onto another area of the page or a separate page.

3.2 Rules for Drawing a Good Flowchart

- Every flowchart must have exactly one clearly marked Start symbol and at least one Stop symbol.
- Flow lines should generally move from top to bottom and left to right, to keep the chart easy to follow.
- Each decision (diamond) symbol must have exactly two or more clearly labeled outgoing paths (e.g., Yes/No or True/False), and every path must eventually lead somewhere — no dangling arrows.
- Only one flow line should enter a process box, though it may have multiple lines converging into a decision or connector symbol.
- Keep each symbol's text short and clear; lengthy explanations should be avoided inside the shapes themselves.

3.3 Example: Flowchart Logic in Words (Check if a Number is Positive or Negative)

The following describes, step by step, what a flowchart for this simple problem would show:

- 1 Start.
- 2 Input a number, N (Parallelogram).

- 3 Decision: Is $N > 0$? (Diamond)
- 4 If Yes $\rightarrow$ Output "Positive" (Parallelogram) $\rightarrow$ go to Stop.
- 5 If No $\rightarrow$ Decision: Is $N < 0$? (Diamond)
- 6 If Yes $\rightarrow$ Output "Negative" $\rightarrow$ go to Stop.
- 7 If No $\rightarrow$ Output "Zero" $\rightarrow$ go to Stop.
- 8 Stop.

4. Dry-Run and its Use

A **dry-run**, also called **desk-checking**, is the process of manually stepping through an algorithm, flowchart, or piece of code on paper — exactly the way a computer would execute it, instruction by instruction — without actually typing it in and running it on a real machine. It is one of the oldest and most reliable techniques for verifying logic before investing time in writing and compiling actual code.

4.1 Why is Dry-Running Useful?

- **Logic Verification:** It helps confirm, with concrete sample inputs, that the algorithm actually produces the correct expected output, rather than just looking correct on a quick read-through.
- **Debugging:** It is one of the most effective ways to catch logical errors (bugs) — mistakes in the reasoning of the algorithm itself — well before the cost and time of compilation and testing.
- **Understanding State:** It forces the programmer to carefully track exactly how the value of every relevant variable changes at each individual step, which is especially valuable for understanding what is happening inside loops.
- **Communication and Review:** A written dry-run/trace table is easy to share with teammates or instructors as evidence that an algorithm behaves correctly, without needing them to read and mentally execute the raw code themselves.

4.2 Example of a Dry-Run Trace Table

Consider the following simple loop, assuming the variable `sum` starts at 0:

```
for (int i = 1; i <= 3; i++) {  
    sum = sum + i;  
}
```

A dry-run trace table lists, step by step, the current value of every relevant variable and condition at each pass through the loop:

Step	Current i	Condition (i <= 3)	Current sum	New sum (sum + i)
1	1	True	0	1
2	2	True	1	3
3	3	True	3	6
4	4	False (Loop Ends)	6	—

By the end of the trace, it becomes clear that the loop correctly computes the sum of the numbers 1 through 3, giving a final result of 6 — confirming that this small piece of logic behaves as intended before it is ever compiled or run.

Practical tip: When dry-running more complex algorithms with multiple variables, nested loops, or conditionals, it is good practice to add one column to the trace table for every variable that can change, plus a column for any condition being tested — this keeps the trace organized and makes it much easier to spot exactly where the logic goes wrong, if it does.

5. Other Logic Development Techniques

Beyond writing plain-language algorithms and drawing flowcharts, programmers rely on several other established techniques to design, document, and communicate logic clearly — especially as the complexity of a program grows.

5.1 Pseudocode

Pseudocode is often described informally as 'fake code'. It expresses the logic of an algorithm using plain, natural-language sentences combined with a loose, informal borrowing of programming-style structural keywords, such as IF, WHILE, FOR, and COMPUTE. Pseudocode deliberately sits exactly between a purely written algorithm and actual, syntactically strict C code — it is structured enough to closely mirror the eventual program's logic, but free enough from strict syntax rules that the programmer can focus entirely on the logic itself, without worrying about semicolons, braces, or data types.

Example:

```
IF student_marks > 40 THEN
    PRINT "Pass"
ELSE
    PRINT "Fail"
END IF
```

Because pseudocode is not tied to any particular programming language's syntax, it is an excellent tool for planning logic that will later be implemented in C, and it also makes it easy to explain an algorithm to someone unfamiliar with C's specific syntax.

5.2 Structure Charts (Top-Down Design)

A structure chart is a visual diagram that shows the overall hierarchical organization of a program. Rather than showing step-by-step logic like a flowchart does, it breaks the main program down into separate functions or modules, and shows how those modules call and interact with one another — typically drawn as a tree, with the main program at the top and its sub-functions branching downward beneath it.

This top-down design approach is extremely well suited to C programming, since C programs are themselves heavily organized around functions: a structure chart makes it easy to plan, at a glance, exactly which functions a C program will need, what each one is responsible for, and how data or control passes between them, before any function bodies are actually written.

Example — a simplified structure chart for a student report-card program (described textually):

```
main()
| -- readStudentData()
| -- calculateTotalMarks()
|   | -- calculateSubjectAverage()
| -- determineGrade()
| -- printReportCard()
```

5.3 Decision Tables

A decision table is a tabular (row-and-column) method used to represent complex logical conditions and the corresponding actions that should be taken for every possible combination of those conditions. It is particularly useful when a problem involves several independent conditions that can combine in many different ways, since a plain else-if ladder or nested if statements can quickly become difficult to read and easy to get wrong for such cases. A decision table forces the designer to explicitly map out every combination, helping ensure that no scenario is accidentally missed.

Example — a decision table for a simple loan-eligibility check based on two conditions:

Has Steady Income?	Good Credit Score?	Action / Decision
Yes	Yes	Approve loan.
Yes	No	Refer for manual review.
No	Yes	Refer for manual review.
No	No	Reject loan.

Notice how the decision table lays out all four possible combinations of the two Yes/No conditions clearly and completely — a structure that could otherwise be easy to get wrong or incomplete if written directly as nested if statements without first planning it out this way.

5.4 Summary: Choosing a Logic Development Technique

Technique	Best Suited For
Algorithm (plain steps)	Quickly outlining the overall logic of a simple, mostly linear process.
Flowchart	Visualizing branching and looping logic clearly, especially for teaching or review.
Pseudocode	Planning logic in a form very close to actual code, without worrying about exact syntax.
Dry-Run / Trace Table	Verifying that an algorithm or piece of code actually behaves correctly before or after coding.
Structure Chart	Planning the overall modular/function-based organization of a larger C program.
Decision Table	Representing and checking complex combinations of multiple independent conditions.

End of Unit — Introduction of Logic Development Tools

Unit 2

Control Structures

Detailed Study Notes — Decision Making, Looping, and Jumping in C

Topics Covered: Selective Control Structures (if, if-else, else-if ladder, switch), the Ternary Operator, Iterative Structures (while, do-while, for), Nested Loops, and Jumping Statements (break, continue, goto).

Introduction: Control Flow in C

By default, a C program executes statements sequentially — one after another, in the exact order they are written, from top to bottom. This is called the **normal flow of control**. However, most real-world problems require more flexibility than a straight sequence of instructions can offer. **Control structures** are language constructs that let a program deviate from this normal sequential flow, allowing it to make decisions, repeat sections of code, or jump to a different point in the program based on specific conditions.

C's control structures fall into three broad categories:

- **Selective (Decision-Making) Structures:** if, if-else, else-if ladder, switch, and the ternary operator — used to choose between alternative paths of execution.
- **Iterative (Looping) Structures:** while, do-while, and for — used to repeat a block of statements multiple times.
- **Jumping Structures:** break, continue, goto, and return — used to transfer control unconditionally to another part of the program.

1. Selective Control Structure

Selective control structures — also called decision-making or conditional statements — allow a program to evaluate a Boolean condition (an expression that is either true, i.e., non-zero, or false, i.e., zero) and then execute one particular block of code out of several possible alternatives.

A. If Statements

The `if` statement is the simplest and most fundamental decision-making structure available in C. It tests a condition and executes a block of statements only when that condition evaluates to true.

Simple if Statement

Executes a block of code only if the specified condition is true. If the condition is false, the block is simply skipped and control moves to the statement immediately following the `if` block.

```
#include <stdio.h>
int main() {
    int age = 20;
    if (age >= 18) {
        printf("You are eligible to vote.\n");
    }
    return 0;
}
```

Output: You are eligible to vote.

Syntax:

```
if (condition) {
    // statements executed only when condition is true
}
```

Note that if the body of the `if` block contains only a single statement, the curly braces `{ }` are technically optional in C — but including them consistently is considered good practice, as it prevents a common class of bugs when the code is later modified to add more statements.

if...else Statement

The `if-else` form provides an alternative block of code that executes automatically whenever the tested condition turns out to be false. Exactly one of the two blocks — never both — will execute for any given run.

```
#include <stdio.h>
int main() {
    int number = 7;
    if (number % 2 == 0) {
        printf("%d is Even.\n", number);
    } else {
        printf("%d is Odd.\n", number);
    }
    return 0;
}
```

Output: 7 is Odd.

else if Ladder

When a program needs to test several mutually exclusive conditions in sequence, the `else-if` ladder chains multiple `if` statements together. Conditions are evaluated from top to bottom, and control jumps to the first block whose condition is true; all remaining conditions in the ladder are then skipped. If none of the conditions match, the final `else` block (if present) acts as a default/fallback.

```
#include <stdio.h>
int main() {
    int marks = 85;
    if (marks >= 90) {
        printf("Grade A\n");
    } else if (marks >= 80) {
        printf("Grade B\n");
    } else if (marks >= 70) {
        printf("Grade C\n");
    } else {
        printf("Fail\n");
    }
    return 0;
}
```

Output: Grade B

Nested if statements: An if or if-else statement can also be placed entirely inside another if or else block. This is called nesting, and it is useful when a second decision only makes sense after the first condition has already been satisfied (for example, checking a person's age only after first confirming they hold a valid ID).

```
if (hasID) {
    if (age >= 18) {
        printf("Entry allowed.\n");
    } else {
        printf("Too young for entry.\n");
    }
} else {
    printf("ID required.\n");
}
```

B. Switch Statement

The switch statement offers a cleaner alternative to a long else-if ladder when a single variable or expression needs to be compared against a series of fixed, exact constant values (integers or characters). It is especially common in menu-driven programs, where the user selects from a numbered list of options.

Key components:

- **switch (expression)** — Evaluates the given expression once, at the top.
- **case value:** — Defines a label to match against the evaluated expression. Execution jumps directly to the matching case.
- **break;** — Immediately exits the switch block. Without it, execution 'falls through' and continues into the next case's statements, regardless of whether that case matches.
- **default:** — An optional block that executes only if none of the case values match the expression; it may be placed anywhere inside the switch, though convention places it last.

```
#include <stdio.h>
int main() {
    int day = 3;
    switch (day) {
        case 1:
            printf("Monday\n");
            break;
        case 2:
            printf("Tuesday\n");
            break;
        case 3:
```

```
        printf("Wednesday\n");
        break;
    default:
        printf("Invalid day\n");
    }
    return 0;
}
```

Output: *Wednesday*

Fall-through behavior: If the break statement is omitted from a case, execution continues into the statements of the next case even though its value does not match. This can occasionally be used deliberately (e.g., to group several cases that should share the same result), but if done accidentally, it is one of the most common sources of bugs in switch-based code.

Restrictions on switch:

- The switch expression must evaluate to an integer or character type (not float, double, or string).
- Each case label must be a constant expression known at compile time — variables are not allowed as case labels.
- No two case labels within the same switch may have the same value.

Comparing if-else ladder and switch

Aspect	else-if Ladder	switch Statement
Type of condition	Can test any kind of relational/logical expression (ranges, multiple variables, etc.)	Can only test equality of a single expression against constant values.
Readability	Becomes harder to read with many conditions.	Very readable for many discrete, fixed options (menus).
Performance	Conditions are checked one at a time, in order.	Compilers can often optimize switch into a fast jump table.

2. Conditional Ternary Operator

The ternary operator, written as `? :`, is C's only operator that takes exactly three operands. It offers a compact, single-line shorthand for a simple if-else statement whenever both branches simply produce a value (rather than performing several separate actions).

Syntax:

```
Condition ? Expression_if_True : Expression_if_False;
```

The condition is evaluated first. If it is true (non-zero), the whole ternary expression takes on the value of `Expression_if_True`, and `Expression_if_False` is never evaluated. If the condition is false (zero), the reverse happens.

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    // Find the maximum of two numbers
    int max = (a > b) ? a : b;

    printf("The maximum number is: %d\n", max);
    return 0;
}
```

Output: The maximum number is: 20

Ternary expressions can also be nested to test multiple conditions, though excessive nesting quickly hurts readability and is generally best avoided beyond one or two levels:

```
int marks = 85;
char *grade = (marks >= 90) ? "A" : (marks >= 80) ? "B" : (marks >= 70) ? "C" : "Fail";
printf("Grade: %s\n", grade);
```

3. Iterative (Looping) Control Statements

Loops allow a block of statements to be executed repeatedly as long as a specified condition remains true, without the programmer having to physically duplicate that block of code. Every loop in C generally requires three components: an **initialization** (setting up a starting value), a **condition** (checked before or after each pass to decide whether to continue), and an **update/increment** (changing the controlling variable so the loop eventually stops).

A. While Loop (Entry-Controlled)

The while loop checks its condition **before** entering the loop body on every pass — this is why it is called an entry-controlled (or pre-tested) loop. If the condition is false right from the start, the loop body will never execute, not even once.

```
#include <stdio.h>
int main() {
    int i = 1;           // Initialization
    while (i <= 3) {     // Condition
        printf("Count: %d\n", i);
        i++;            // Increment/Update
    }
    return 0;
}
```

Output: Count: 1 Count: 2 Count: 3

Caution: If the update step is forgotten, or the condition never becomes false, the loop will run forever — a bug known as an **infinite loop**.

B. Do...while Loop (Exit-Controlled)

The do...while loop checks its condition **after** executing the loop body — making it exit-controlled (or post-tested). This guarantees the loop body always runs at least once, regardless of whether the condition is true or false at the start.

```
#include <stdio.h>
int main() {
    int i = 5;
    do {
        printf("Value: %d\n", i);
        i++;
    } while (i < 3);    // Condition is checked at the end
    return 0;
}
```

Output: Value: 5 (Executes once, since the check only happens afterward)

Note the required semicolon after the while(condition) in a do-while loop — this is a common syntax mistake for beginners, since a plain while loop does not need one after its condition.

C. For Loop (Entry-Controlled)

The for loop is ideal whenever the exact number of iterations is known (or can be calculated) in advance. It is compact because it consolidates all three loop components — initialization, condition, and update — into a single header line.

```
#include <stdio.h>
int main() {
    // for (initialization; condition; increment/decrement)
```

```
for (int i = 1; i <= 3; i++) {  
    printf("Iteration: %d\n", i);  
}  
return 0;  
}
```

Output: Iteration: 1 Iteration: 2 Iteration: 3

Execution order of a for loop:

- 1. The initialization expression runs exactly once, before the loop starts.
- 2. The condition is checked; if false, the loop ends immediately.
- 3. If the condition is true, the loop body executes.
- 4. The update expression runs.
- 5. Control returns to step 2, and the cycle repeats.

Any of the three parts of a for loop's header may be omitted (though the semicolons must remain), and multiple variables can be initialized/updated using the comma operator, e.g. `for (i = 0, j = 10; i < j; i++, j--)`.

Comparing the Three Loop Types

Loop	Condition Checked	Minimum Executions	Best Used When
while	Before the body (entry-controlled)	0	The number of iterations is not known in advance.
do...while	After the body (exit-controlled)	1	The body must run at least once regardless of the condition (e.g., menu display).
for	Before the body (entry-controlled)	0	The exact number of iterations is known or countable in advance.

4. Nesting of Loops

Nesting means placing one loop entirely inside the body of another loop. The loop on the outside is called the **outer loop**, and the loop inside it is the **inner loop**. For every single iteration of the outer loop, the inner loop runs through *all* of its own iterations before control returns to the outer loop for its next pass.

Nested loops are heavily used for working with multi-dimensional data (such as matrices or grids), generating patterns (like triangles of stars or numbers), and any task that requires comparing or combining every element of one set with every element of another.

```
#include <stdio.h>
int main() {
    // Outer loop controls rows
    for (int i = 1; i <= 3; i++) {
        // Inner loop controls columns
        for (int j = 1; j <= 2; j++) {
            printf("Row %d, Col %d | ", i, j);
        }
        printf("\n"); // Move to next line after inner loop finishes
    }
    return 0;
}
```

Output:

```
Row 1, Col 1 | Row 1, Col 2 |
Row 2, Col 1 | Row 2, Col 2 |
Row 3, Col 1 | Row 3, Col 2 |
```

Total iterations: If the outer loop runs m times and the inner loop runs n times for each outer pass, the inner loop's body executes a total of $m \times n$ times overall. This is important to keep in mind for performance, since deeply nested loops can grow very expensive very quickly as the data size increases.

Example — printing a right-angled triangle pattern using nested loops:

```
#include <stdio.h>
int main() {
    int rows = 4;
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
/* Output:
*
**
***
****
*/
```

5. Jumping Statements

Jumping statements transfer control of the program to a different location, unconditionally, interrupting the normal sequential or looping flow. C provides four such statements: `break`, `continue`, `goto`, and `return` (`return` is covered in detail with functions, since it exits a function rather than just a loop).

A. Break Statement

The `break` statement is used to immediately exit the innermost enclosing loop (`while`, `do-while`, or `for`) or a `switch` statement, bypassing any remaining iterations or statements in that block. Execution resumes at the very next statement after the loop or `switch`.

```
#include <stdio.h>
int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            break; // Loop terminates completely when i is 3
        }
        printf("%d ", i);
    }
    return 0;
}
```

Output: 1 2

In nested loops, a `break` statement only exits the **innermost** loop that directly contains it — the outer loop(s) continue running normally.

B. Continue Statement

The `continue` statement skips the remaining statements in the **current** iteration of the loop and jumps directly to the next iteration — it does *not* terminate the loop as a whole. In a `for` loop, the update expression still runs before the next condition check; in a `while` loop, control returns straight to the condition check.

```
#include <stdio.h>
int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            continue; // Skips the rest of the code for i=3
        }
        printf("%d ", i);
    }
    return 0;
}
```

Output: 1 2 4 5

C. Goto Statement

The `goto` statement allows a program to jump directly to any labeled statement within the same function, regardless of the normal flow of control — forward or backward, into or out of loops.

Note: Using `goto` is generally discouraged in modern programming because uncontrolled jumps make code very difficult to read, trace, and maintain — a phenomenon often nicknamed 'spaghetti code'. Nearly any logic written with `goto` can be rewritten more clearly using structured loops and conditionals instead. It remains part of the C language and does have a few legitimate niche uses, such as breaking out of deeply nested loops in one step, or handling centralized cleanup/error-exit code near the end of a function.

```
#include <stdio.h>
int main() {
    int num = 1;

    start_label:          // This is a label
    printf("%d ", num);
    num++;

    if (num <= 3) {
        goto start_label; // Jumps back to the label
    }

    return 0;
}
```

Output: 1 2 3

Summary: break vs continue vs goto

Statement	Effect	Scope
break	Exits the loop/switch entirely.	Innermost enclosing loop or switch only.
continue	Skips to the next iteration of the loop.	Innermost enclosing loop only.
goto	Jumps unconditionally to a labeled statement.	Anywhere within the same function.

End of Unit 2 — Control Structures

Unit 3

Functions (Inbuilt and User Defined)

Detailed Study Notes — Library Functions, User-Defined Functions, Parameter Passing, Recursion, and Storage Classes

Topics Covered: Library (Inbuilt) Functions across standard headers, User-Defined Function categories, Call by Value vs Call by Reference, Recursion, Storage Classes, and Passing/Returning Values.

Introduction to Functions

A **function** is a self-contained block of code, given a name, that is designed to perform a specific, well-defined task. Instead of writing the same set of instructions repeatedly throughout a program, a function lets that logic be written once and then invoked (called) as many times as needed, from anywhere in the program.

Why use functions?

- **Modularity:** A large, complex program can be broken down into smaller, independent, and more manageable pieces, each handling one specific task.
- **Reusability:** Once written and tested, a function can be called repeatedly — from multiple places within the same program, or even reused in other programs — without rewriting its logic.
- **Easier Debugging and Maintenance:** Since each function has a clear, isolated responsibility, errors are easier to trace to their source, and changes to one part of the logic can be made in a single place.
- **Abstraction:** A function's caller only needs to know what the function does and how to call it (its interface), not the internal details of how it accomplishes the task.

Every function in C has every one of the following essential parts:

- **Function Declaration (Prototype):** Announces the function's name, return type, and parameter types to the compiler before it is used.
- **Function Definition:** Contains the actual body — the block of statements that execute when the function is called.
- **Function Call:** The statement elsewhere in the program that invokes the function by name, optionally passing it arguments.

C provides two broad categories of functions: **Library (Inbuilt) Functions**, which are pre-written, pre-compiled functions supplied as part of the C standard library (such as `printf()` and `sqrt()`), and **User-Defined Functions**, which are functions that the programmer writes to perform tasks specific to their own program.

1. Types of Library Functions

Library functions are ready-made functions provided by the C standard library, already written, tested, and compiled by the compiler vendor. To use them, a program must `#include` the corresponding header file at the top of the source file, which supplies the function's declaration (prototype) so the compiler knows how the function should be called.

A. String Functions (<string.h>)

These functions are used for manipulating strings — arrays of characters terminated by a null character `'\0'`.

Function	Description
<code>strlen(s)</code>	Returns the length of the string <code>s</code> , not counting the null terminator.
<code>strcpy(s1, s2)</code> / <code>strncpy(s1, s2, n)</code>	Copies string <code>s2</code> into <code>s1</code> . <code>strncpy</code> copies at most <code>n</code> characters, which is safer against buffer overflow.
<code>strcat(s1, s2)</code> / <code>strncat(s1, s2, n)</code>	Concatenates (appends) <code>s2</code> onto the end of <code>s1</code> . <code>strncat</code> appends at most <code>n</code> characters from <code>s2</code> .
<code>strcmp(s1, s2)</code> / <code>strncmp(s1, s2, n)</code>	Compares two strings character by character. Returns 0 if identical, a negative value if <code>s1 < s2</code> , a positive value if <code>s1 > s2</code> .
<code>strchr(s, c)</code> / <code>strrchr(s, c)</code>	Finds the first / last occurrence of character <code>c</code> within string <code>s</code> , returning a pointer to it (or NULL if not found).
<code>strstr(s1, s2)</code>	Finds the first occurrence of the substring <code>s2</code> within string <code>s1</code> , returning a pointer to that location.
<code>strspn</code> , <code>strcspn</code> , <code>strpbrk</code> , <code>strtok</code>	More advanced functions for spanning character sets and tokenizing (splitting) a string into pieces using delimiters.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[20] = "Hello ";
    char str2[] = "World";

    printf("Length of str2: %lu\n", strlen(str2));

    strcat(str1, str2);
    printf("Concatenated String: %s\n", str1);

    return 0;
}
```

Output: Length of str2: 5 | Concatenated String: Hello World

Caution: Functions like `strcpy` and `strcat` do not check whether the destination array is large enough to hold the result. If the destination buffer is too small, this can cause a buffer overflow — writing past the end of the array into adjacent memory. Prefer the bounded versions (`strncpy`, `strncat`) or ensure destination arrays are large enough.

B. Mathematical Functions (<math.h>)

These functions perform common mathematical calculations. Most `math.h` functions take and return values of type `double` (double-precision floating point), regardless of the type of the argument passed in.

Category	Functions	Description
Trigonometry	sin(x), cos(x), tan(x), asin(x), acos(x), atan(x)	Standard and inverse trigonometric functions (angles in radians).
Powers & Roots	pow(x, y), sqrt(x), exp(x)	pow(x, y) returns x raised to the power y; sqrt(x) returns the square root; exp(x) returns e raised to the power x.
Rounding	ceil(x), floor(x)	ceil(x) rounds up to the nearest integer value; floor(x) rounds down.
Absolute Value	fabs(x)	Returns the absolute (positive) value of a floating-point number.
Division & Remainders	fmod(x, y), modf(), div()	fmod(x, y) gives the floating-point remainder of x/y; modf() splits a value into its integer and fractional parts; div() performs integer division and returns both quotient and remainder.
Logarithms	log(x), log10(x)	log(x) returns the natural logarithm (base e); log10(x) returns the base-10 logarithm.

```
#include <stdio.h>
#include <math.h>

int main() {
    double base = 2.0, exponent = 3.0;
    printf("pow: %.2f\n", pow(base, exponent));    // 8.00
    printf("sqrt: %.2f\n", sqrt(16.0));           // 4.00
    printf("ceil: %.2f\n", ceil(4.1));             // 5.00
    printf("floor: %.2f\n", floor(4.9));           // 4.00
    return 0;
}
```

Note: Programs using math.h functions often need to be linked with the math library explicitly on some compilers/platforms (e.g., using the -lm flag with GCC on Linux).

C. I/O Formatting Functions (<stdio.h>)

These functions handle standard input and output — reading data from and writing data to the console (or files).

Function	Type	Description
printf() / scanf()	Formatted	Prints formatted output to / reads formatted input from the standard console using format specifiers such as %d, %c, %s, %f.
gets() / puts()	String	Reads / prints a full string (a line of text). Note: gets() is generally considered unsafe (no bounds checking) and has been removed from the C11 standard; modern C code prefers fgets().
getchar() / putchar()	Character	Reads / prints a single character from/to the standard console.
getc() / putc()	File/Stream	Similar to getchar/putchar, but can operate on any specified stream, including files opened with fopen().
ungetc()	Character	Pushes a character back onto an input stream, so that it will be the next character read.

Why gets() is unsafe: gets() reads characters into a buffer until a newline, with no way to specify a maximum length — if the input line is longer than the buffer, it silently writes past the end of the array. fgets(buffer, size, stdin) solves this by requiring the buffer size as an explicit argument.

D. Miscellaneous Character Functions (<ctype.h>)

These functions test or convert individual characters. The testing functions return a non-zero (true) value if the condition holds, or zero (false) otherwise.

- **Checks:** isalnum() (is alphanumeric?), isalpha() (is an alphabet letter?), isdigit() (is a digit 0-9?), islower() / isupper() (is lowercase/uppercase?), isspace() (is a blank/whitespace character?), isxdigit() (is a valid hexadecimal digit?).
- **Conversions:** toupper() converts a lowercase letter to uppercase; tolower() converts an uppercase letter to lowercase. Non-alphabetic characters are returned unchanged.

Older/legacy note: delay() and clrscr() are functions from the older Turbo C <conio.h>/<dos.h> headers, used respectively to pause execution for a given time and to clear the console screen — they are not part of the standard C library and are largely obsolete on modern compilers. errno, declared in <errno.h>, is a global variable used by several library functions to report the specific reason a runtime error occurred.

E. Standard Library Functions (<stdlib.h>)

A collection of general-purpose utility functions covering numeric conversions, program control, and randomness.

Category	Functions	Description
Math	abs(), labs()	abs() returns the absolute value of an int; labs() does the same for a long int.
Conversion	atof(), atol(), strtoul()	atof() converts a string to a float/double; atol() converts a string to a long; strtoul() converts a string to an unsigned long, with more robust error handling.

Category	Functions	Description
System	exit()	Terminates the program immediately, optionally returning a status code to the operating system.
Randomness	rand(), srand()	rand() generates a pseudo-random integer; srand() seeds the random number generator (commonly seeded once with the current time so results differ between runs).

F. Memory Allocation Functions (<stdlib.h>)

These functions support **dynamic memory allocation** — reserving and releasing memory explicitly at runtime, rather than relying only on memory automatically managed at compile time. This is essential when the amount of memory needed is not known until the program is actually running (e.g., an array whose size depends on user input).

Function	Description
malloc(size)	Allocates a single block of memory of the specified size, in bytes. The allocated memory is not initialized and contains garbage (unpredictable) values.
calloc(n, size)	Allocates memory for n blocks, each of the given size, and initializes every byte of that memory to zero.
realloc(ptr, new_size)	Resizes a block of memory previously allocated by malloc/calloc/realloc to a new size, preserving existing contents up to the smaller of the old and new sizes.
free(ptr)	Releases (deallocates) memory that was previously allocated dynamically, returning it to the system. Failing to call free() when memory is no longer needed causes a memory leak.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = (int *)malloc(n * sizeof(int));    // allocate space for 5 ints

    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        arr[i] = i * i;
    }

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);    // release memory once done
    return 0;
}
```

Output: 0 1 4 9 16

Best practice: Always check whether malloc/calloc returned NULL (indicating the allocation failed, e.g. due to insufficient memory) before using the pointer, and always call free() exactly once for every successful allocation to avoid memory leaks or double-free errors.

2. Types of User-Defined Functions

Functions written by the programmer — as opposed to library functions — are called user-defined functions. Every user-defined function can be classified along two independent dimensions: whether it accepts any arguments (parameters), and whether it returns a value back to its caller. This gives four possible combinations:

Category	Description	Example Signature
No Arguments, No Return Value	Performs a fixed task with no input and no output value; typically used for simple, repeated actions like printing a fixed message.	<code>void printMessage() { ... }</code>
No Arguments, Returns a Value	Computes and returns a value without needing any input from the caller.	<code>int getYear() { return 2024; }</code>
Arguments, No Return Value	Accepts input data to work with, but only produces a side effect (like printing), without sending any value back.	<code>void printSum(int a, int b) { ... }</code>
Arguments, Returns a Value	The most flexible and common form: takes input data and computes/returns a result based on it.	<code>int add(int a, int b) { return a + b; }</code>

Choosing the right category for a given task depends on what the function needs to know (its inputs) and what, if anything, the calling code needs back from it (its output).

3. Function Call by Value vs. Call by Reference

This is one of the most important concepts to understand about how data moves between a function and its caller in C — it determines whether changes made inside a function are visible to the code that called it.

A. Call by Value

Concept: When an argument is passed by value, the function receives a completely independent *copy* of the actual variable's value, stored in its own separate memory location (the function's local parameter).

Effect: Because the function only ever works with a copy, any changes made to the parameter inside the function have no effect whatsoever on the original variable back in the calling function. Once the function returns, the copy is discarded and the original variable is completely unaffected.

```
#include <stdio.h>
void addTen(int x) {
    x = x + 10;    // Changes only the local copy
}
int main() {
    int num = 5;
    addTen(num);
    printf("Value is: %d\n", num);    // Output is still 5
    return 0;
}
```

Output: Value is: 5 (the original 'num' is unchanged)

B. Call by Reference

Concept: When an argument is passed by reference, instead of a copy of the value, the function is given the memory address (a pointer) of the actual original variable, obtained using the address-of operator &.

Effect: Because the function now has direct access to the exact memory location of the original variable, any changes made inside the function by dereferencing the pointer (*x) directly modify the original variable itself — these changes remain visible to the calling function even after the called function has returned.

```
#include <stdio.h>
void addTen(int *x) {
    *x = *x + 10;    // Dereferences the pointer to change the actual value
}
int main() {
    int num = 5;
    addTen(&num);    // Passing the memory address using &
    printf("Value is: %d\n", num);    // Output is now 15
    return 0;
}
```

Output: Value is: 15 (the original 'num' was directly modified)

Comparison: Call by Value vs Call by Reference

Aspect	Call by Value	Call by Reference
What is passed	A copy of the actual value.	The memory address (pointer) of the original variable.
Effect on original variable	No effect — original remains unchanged.	Original variable is directly modified.

Aspect	Call by Value	Call by Reference
Memory usage	Extra memory used for the copy.	No extra copy of the data itself, only the address.
Syntax cue	Plain parameter, e.g. int x	Pointer parameter, e.g. int *x, called with &variable
Typical use	When the function only needs to read/use a value.	When the function needs to modify the caller's variable, or return multiple values.

Important note on arrays: Arrays in C are always effectively passed by reference, even without explicitly using a pointer syntax at the call site — when an array name is passed as an argument, it decays into a pointer to its first element, so changes made to array elements inside a function are visible in the caller.

4. Recursion

Recursion occurs when a function calls itself, either directly (calling its own name inside its own body) or indirectly (calling another function, which in turn calls the original function), in order to solve a smaller instance of the very same problem.

Crucial requirement — the base condition: Every recursive function must include at least one **base condition** (also called a base case) that stops the recursive calls under some specific circumstance. Without a base condition — or if the recursive calls never actually reach it — the function will keep calling itself indefinitely, rapidly consuming memory on the call stack, until the program crashes with a **stack overflow** error.

Example — Calculating Factorial using Recursion ($5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$):

```
#include <stdio.h>

int factorial(int n) {
    // Base condition
    if (n == 0 || n == 1) {
        return 1;
    }
    // Recursive call
    return n * factorial(n - 1);
}

int main() {
    printf("Factorial of 5 is: %d\n", factorial(5));
    return 0;
}
```

Output: Factorial of 5 is: 120

How the recursive calls unfold (call stack trace):

```
factorial(5)
= 5 * factorial(4)
= 5 * (4 * factorial(3))
= 5 * (4 * (3 * factorial(2)))
= 5 * (4 * (3 * (2 * factorial(1))))
= 5 * (4 * (3 * (2 * 1)))          <- base case returns 1
= 5 * 4 * 3 * 2 * 1
= 120
```

Each call to `factorial()` waits, paused, on the call stack for the result of the next call to come back, before it can compute and return its own result. Once `factorial(1)` hits the base case and returns 1, the chain of waiting calls unwinds back up the stack, multiplying at each level, until the original call finally returns 120.

Recursion vs Iteration:

- Recursive solutions are often shorter and can express certain problems (like tree traversal, or mathematically defined sequences) more naturally and elegantly than loops.
- However, recursion generally uses more memory than an equivalent loop, since every nested call adds a new frame to the call stack until the base case is reached.
- Any recursive algorithm can, in principle, be rewritten using iteration (loops) instead, though sometimes at the cost of a less intuitive implementation.

5. Storage Classes

A storage class in C defines three key properties of a variable: its **scope** (where in the program the variable is visible and can be accessed), its **lifetime** (how long the variable continues to exist in memory), and its **default initial value** if the programmer does not explicitly initialize it.

Storage Class	Keyword	Scope	Lifetime	Initial Value
Automatic	auto	Local (inside the block/function where declared)	Ends when the block/function exits	Garbage (indeterminate)
Register	register	Local	Ends when block exits (requested to be stored in a CPU register for faster access)	Garbage
Static	static	Local (but the variable retains its value between calls)	Exists for the entire duration of the program	Zero
External	extern	Global (can be shared across multiple source files)	Exists for the entire duration of the program	Zero

Notes on Each Storage Class

- **auto:** This is the default storage class for any variable declared inside a function or block if no other storage class is specified — most ordinary local variables are automatic, whether or not the keyword is written explicitly.
- **register:** A hint to the compiler that a variable will be accessed very frequently and should, if possible, be kept in a CPU register rather than in main memory for faster access. Modern compilers generally optimize this automatically and may ignore the hint; a register variable's address cannot be taken with &.
- **static:** A static local variable is initialized only once, the very first time the function is called, and it then retains its last value between successive calls to that function — unlike an ordinary automatic variable, which is recreated fresh (and loses its previous value) every time the function is entered. A static variable declared outside any function instead limits that variable's visibility to just the file it is declared in.
- **extern:** Used to declare a variable that is actually defined in a different source file (or later in the same file), allowing that single variable to be shared and accessed across multiple files in a multi-file program.

Example of static

```
void counter() {
    static int count = 0;    // Initialized only once, on the first call
    count++;
    printf("%d ", count);
}

int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

Calling counter() three times outputs: 1 2 3 (not 1 1 1, because 'count' persists between calls)

6. Passing and Returning Values

6.1 Passing Values

Data is passed into a function by placing arguments inside the parentheses of the function call, for example `calculateArea(radius)`. These arguments map, in order, onto the function's declared parameters. C allows a wide variety of data to be passed as arguments:

- **Basic data types** such as `int`, `float`, `char`, and `double` — passed by value by default.
- **Entire arrays** — which are always passed effectively by reference, since an array name decays into a pointer to its first element when passed to a function.
- **Structures** — an entire struct can be passed by value (a full copy is made) or by passing a pointer to it (call by reference).
- **Pointers** — allowing the function to directly access and modify the caller's original data, or to work with dynamically allocated memory.

6.2 Returning Values

A function can send exactly **one** value back to its caller using the `return` statement. The data type of the value being returned must match (or be automatically convertible to) the function's declared return type.

- If a function is declared with a specific return type (e.g., `int`, `float`, `char`), it must use `return` to send back a value of that type.
- If a function does not need to send any value back to its caller, its return type is declared as `void`, and a bare `return`; statement (with no value) may optionally be used to exit the function early.
- Although a function can only directly return one value, multiple results can effectively be 'returned' by instead passing pointers to several variables as arguments (call by reference) and modifying them inside the function.

```
#include <stdio.h>

// Returning multiple results via pointer parameters
void minMax(int a, int b, int *minOut, int *maxOut) {
    if (a < b) {
        *minOut = a;
        *maxOut = b;
    } else {
        *minOut = b;
        *maxOut = a;
    }
}

int main() {
    int small, large;
    minMax(15, 7, &small, &large);
    printf("Min: %d, Max: %d\n", small, large);
    return 0;
}
```

Output: Min: 7, Max: 15

Unit 4

Arrays and Pointers

Detailed Study Notes — Single, Multi-Dimensional, and String Arrays; Pointers, Pointer Arithmetic, and Structures

Topics Covered: Arrays (1D, 2D, Multi-dimensional, String Arrays), Uses of Arrays, Matrix Operations, Pointers (Address-of and Dereference), Dynamic Programming with Pointers, Pointer Arithmetic, Arrays of Pointers, Pointers to Structures, Pointer-to-Pointer, and the Dangling Pointer Problem.

1. Array

An **array** is a collection of elements, all of the same data type, stored together in contiguous (back-to-back, side-by-side) memory locations. Rather than declaring many separate variables — such as num1, num2, num3 — for a set of related values, all of them can be stored under a single variable name using an array, e.g. num[3].

Key characteristics of arrays in C:

- All elements must be of the **same data type** and therefore occupy the same amount of memory each.
- Elements are stored in **contiguous memory**, which allows extremely fast, direct access to any element using its index.
- Indexing is **zero-based** — the first element is at index 0, and the last element of an array of size n is at index n-1.
- The size of a standard (static) array must generally be known at compile time and cannot be changed once declared.
- Accessing an index outside the valid range (0 to n-1) is called an **out-of-bounds access**; C does not automatically check for this, so it can silently corrupt other memory or crash the program.

Types of Arrays

Single Dimensional Array

The simplest and most common form of array, representing a linear (single-row) list of items. Each element is accessed using exactly one index, counting from 0.

```
#include <stdio.h>
int main() {
    int marks[3] = {85, 90, 78};    // Declaration and initialization
    printf("First student marks: %d\n", marks[0]);    // Accessing the first element
    return 0;
}
```

Output: First student marks: 85

Two Dimensional Array

A 2D array can be thought of as an 'array of arrays' — it is commonly used to represent tables, grids, or mathematical matrices organized into rows and columns. Accessing any element requires **two indices**: array[row][column].

```
#include <stdio.h>
int main() {
    // A 2x3 matrix (2 rows, 3 columns)
    int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} };
    printf("Value at row 1, col 2: %d\n", matrix[1][2]);
    return 0;
}
```

Output: Value at row 1, col 2: 6

Memory layout: Internally, C stores a 2D array in **row-major order** — meaning all the elements of row 0 are stored first in memory, immediately followed by all elements of row 1, and so on. This is important to understand when working with pointer arithmetic on 2D arrays.

Multi-dimensional Array

C also supports arrays with three or more dimensions. A 3D array can be thought of conceptually as an array of 2D arrays — like a stack of tables, or a cube of data — useful for representing things such as multiple

frames of a 2D grid over time, or 3D spatial data.

```
// Declaration of a 3D array (2 blocks, 3 rows, 4 columns)
int cube[2][3][4];
```

The total number of elements in a multi-dimensional array is simply the product of all its dimension sizes — the example above holds $2 \times 3 \times 4 = 24$ integers in total.

String Arrays

Recall that a single string in C is itself a 1D array of characters, terminated by a null character. Therefore, an **array of strings** is naturally implemented as a 2D array of characters, where each row represents one complete string.

```
#include <stdio.h>
int main() {
    char days[3][10] = {"Monday", "Tuesday", "Wednesday"};
    printf("Day 2 is: %s\n", days[1]);
    return 0;
}
```

Output: Day 2 is: Tuesday

In `char days[3][10]`, each of the 3 rows can hold a string of up to 9 characters plus its null terminator. Note that this fixed-width approach can waste memory when strings are of very different lengths — the 'Array of Pointers' technique covered later in this unit offers a more memory-efficient alternative for storing variable-length strings.

Use of Arrays in Programming

- **Data Storage:** Storing multiple related values of the same type under a single variable name, rather than juggling many separately named variables.
- **Code Optimization:** Reducing the number of variables needed and writing cleaner, shorter code by using loops to iterate through elements, instead of repeating near-identical statements for each variable.
- **Data Structures:** Arrays serve as the fundamental building block for constructing more complex data structures, such as stacks, queues, and hash tables.
- **Sorting and Searching:** Arrays are essential for implementing classic algorithms — such as bubble sort, binary search, and linear search — that operate over collections of data.

Arrays and Matrices

In C, two-dimensional arrays are the standard way to represent and perform mathematical matrix operations — such as addition, multiplication, and transposing — since a matrix's row-and-column structure maps directly onto a 2D array's [row][column] indexing.

```
#include <stdio.h>
int main() {
    int a[2][2] = {{1, 2}, {3, 4}};
    int b[2][2] = {{5, 6}, {7, 8}};
    int sum[2][2];

    // Matrix Addition
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            sum[i][j] = a[i][j] + b[i][j];
            printf("%d ", sum[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

6 8
10 12

Matrix multiplication (conceptual note): Unlike addition, matrix multiplication requires a *third* nested loop, since each element of the result matrix is computed as the sum of products across an entire row of the first matrix and an entire column of the second — this is typically covered in more depth alongside nested loops and functions.

2. Pointers

Introduction of Pointers

A **pointer** is a special kind of variable that stores the memory address of another variable, rather than storing that variable's actual data directly. Because a pointer gives direct access to a specific location in memory, pointers are extremely powerful tools that enable efficient memory manipulation, dynamic memory allocation, and the construction of complex linked data structures.

- **& (Address-of operator):** When placed before a variable name, returns the memory address at which that variable is stored.
- *** (Dereference operator):** When placed before a pointer variable, accesses (reads or writes) the actual value stored at the memory address the pointer is currently holding.
- **Pointer declaration:** A pointer must be declared to point to a specific data type, e.g. `int *ptr;` declares a pointer that can only validly store the address of an int variable.

Use of Pointers in Dynamic Programming

- **Dynamic Memory Allocation:** Functions such as `malloc()` and `calloc()` return pointers to memory blocks that are allocated manually, at runtime, rather than being fixed at compile time.
- **Efficiency:** Passing large arrays or structures to a function via a pointer avoids the overhead of copying the entire dataset — the function instead works directly with the original data through its address (an implicit form of call by reference).
- **Complex Data Structures:** Pointers are essential for building dynamic structures such as linked lists, trees, and graphs, where individual data 'nodes' are connected to one another via pointers rather than sitting in one fixed, contiguous block.

Pointer to Variables

This is the most basic use of a pointer: assigning the address of an ordinary variable of a standard data type to it.

```
#include <stdio.h>
int main() {
    int num = 10;
    int *ptr = &num;    // ptr now holds the address of num

    printf("Value of num: %d\n", *ptr);
    printf("Address of num: %p\n", ptr);
    return 0;
}
```

Output: Value of num: 10 | Address of num: (some memory address, e.g. 0x7ffd5...)

Pointer to Array

In C, the plain name of an array, when used by itself, decays into (behaves as) a constant pointer to its very first element. This is why an array can be assigned directly to a pointer of the matching type, without needing the address-of operator.

```
int arr[5] = {10, 20, 30};
int *p = arr;    // Equivalent to: int *p = &arr[0];
```

Pointer within Array (Pointer Arithmetic)

A pointer can be moved forward or backward through the elements of an array by incrementing or decrementing it. Importantly, when a value is added to a pointer, C automatically scales that addition by the size of the pointer's data type — so adding 1 to an `int *` pointer moves it forward by `sizeof(int)` bytes (typically 4), landing exactly on the next integer element, not just one raw byte forward.

```
#include <stdio.h>
int main() {
    int arr[3] = {100, 200, 300};
    int *ptr = arr;    // Points to 100

    printf("First element: %d\n", *ptr);

    ptr++;    // Moves pointer to the next integer address
    printf("Second element: %d\n", *ptr);

    return 0;
}
```

Output: First element: 100 | Second element: 200

Common pointer arithmetic operations: `ptr++` / `ptr--` move to the next/previous element; `ptr + n` computes the address `n` elements ahead without modifying `ptr` itself; and subtracting one pointer from another (`ptr2 - ptr1`), when both point into the same array, gives the number of elements between them.

Array of Pointer

An array of pointers is an array in which every single element is itself a pointer, rather than a plain data value. This technique is especially useful for creating collections of variable-length strings, since each pointer can point to a string of exactly the length it needs — avoiding the wasted memory of a fixed-width 2D character array where every row must be as wide as the longest possible string.

```
#include <stdio.h>
int main() {
    // Array of pointers to strings
    char *names[] = {"Alice", "Bob", "Charlie"};

    for (int i = 0; i < 3; i++) {
        printf("%s\n", names[i]);
    }
    return 0;
}
```

Output:

Alice

Bob

Charlie

Pointer To Structure

When a pointer is used to access the members of a structure, C requires the **arrow operator** (`->`) instead of the standard dot operator (`.`) that is used when accessing members directly through a non-pointer structure variable.

```
#include <stdio.h>
struct Student {
    int id;
};
```

```
int main() {  
    struct Student s1 = {101};  
    struct Student *ptr = &s1;  
  
    // Accessing member using arrow operator  
    printf("Student ID: %d\n", ptr->id);  
    return 0;  
}
```

Output: Student ID: 101

Note that `ptr->id` is simply a more convenient shorthand for `(*ptr).id` — first dereferencing the pointer to get the actual structure, then accessing its member with the dot operator. Both forms are equivalent, but the arrow operator is far more commonly used in practice.

Pointers within Structure

A structure can also have a pointer as one of its own internal members — this is common when a structure needs to reference a variable-length piece of data (like a string) or needs to link to another structure, as is the case in linked lists.

```
struct Employee {
    int empId;
    char *name;    // Pointer within a structure
};
```

Pointer to Pointer (Double Pointer)

A pointer to a pointer is a variable that stores the memory address of *another pointer*, rather than the address of an ordinary data variable. It is declared using multiple asterisks — one asterisk for each additional level of indirection (e.g. `int **ptr2`; for a pointer to an int pointer).

```
#include <stdio.h>
int main() {
    int var = 50;
    int *ptr1 = &var;    // Pointer to var
    int **ptr2 = &ptr1;  // Pointer to pointer ptr1

    printf("Value using double pointer: %d\n", **ptr2);
    return 0;
}
```

Output: Value using double pointer: 50

Here, `*ptr2` dereferences once to retrieve the value stored inside `ptr1`, which is the address stored in `ptr1` — giving back `ptr1` itself. Dereferencing a second time, `**ptr2`, follows that address on to the original variable `var`, yielding its value, 50. Double pointers are commonly used when a function needs to modify the value of a pointer itself (not just the data it points to), and in dynamically allocated 2D arrays and arrays of strings.

Dangling Pointer Problem

A **dangling pointer** is a pointer that continues to hold the address of a memory location that has already been freed (deallocated), deleted, or has otherwise gone out of scope, even though the pointer variable itself still exists and appears to be valid. Accessing (reading from or writing to) a dangling pointer results in **undefined behavior** — the program might crash immediately, silently corrupt unrelated data, or appear to work correctly by coincidence, only to fail unpredictably later.

Example of causing a dangling pointer:

```
#include <stdlib.h>
int main() {
    int *ptr = (int *)malloc(sizeof(int));
    *ptr = 10;

    free(ptr);    // Memory is deallocated

    // ptr is now a Dangling Pointer!
    // Best practice to fix this:
    ptr = NULL;

    return 0;
}
```

Common causes of dangling pointers:

- Freeing dynamically allocated memory (with `free()`) while a pointer still refers to that address, and then continuing to use that pointer.
- Returning the address of a local variable from a function — once the function returns, that variable's stack memory is no longer valid, so any pointer to it becomes dangling.
- Letting two or more pointers refer to the same memory, freeing that memory through one pointer, and then still using the other pointer as though it were valid.

Best practice: Immediately after freeing a pointer with `free()`, explicitly set that pointer to `NULL`. Since dereferencing a `NULL` pointer causes an immediate, easily detectable crash rather than silent memory corruption, this practice turns a dangerous, hard-to-trace bug into one that is much easier to catch and debug.

End of Unit 4 — Arrays and Pointers

Unit 5

User Defined Data Type – Structure, Union & enum

Detailed Study Notes — Declaring, Initializing, and Using Structures, Unions, and Enumerations in C

Topics Covered: Structures (declaration, initialization, dynamic allocation, pointers, arrays, functions, nesting), Unions and how they differ from Structures, and Enumerated Types.

Introduction

Basic (primary) data types such as `int`, `float`, and `char` can each store only a single value, and arrays extend this to store multiple values, but only of the *same* type. Real-world data, however, is often more complex — a single 'record' frequently needs to combine several *different* kinds of related data together (for example, a student's numeric roll number, textual name, and floating-point marks, all as one unit). To model this kind of data cleanly, C provides three **user-defined data types**: the **structure**, the **union**, and the **enumeration**.

1. What is a Structure?

A **structure** (declared using the `struct` keyword) is a user-defined data type in C that allows several data items of different types to be grouped together and referred to under a single name. It is particularly well suited to representing a **record** — a single logical unit of related information, such as a 'Student' record made up of an integer ID, a character array for the name, and a float for the marks.

Once a structure type is defined, it behaves much like any other data type in C: variables of that structure type can be declared, initialized, assigned, passed to functions, stored in arrays, and pointed to — all of which are covered throughout this unit.

2. Initializations and Declarations

Before a structure can actually be used, its 'blueprint' must first be defined — this tells the compiler what members the structure contains and what type each member is. Only after this blueprint exists can variables of that structure type be declared.

Declaration (Defining the Blueprint)

```
struct Book {
    int book_id;
    char title[50];
    float price;
}; // Notice the semicolon at the end!
```

This declaration alone does not create any variable or reserve any memory — it simply teaches the compiler what a 'struct Book' looks like. The trailing semicolon after the closing brace is required and is a very common point where beginners make a syntax mistake.

Initialization (Creating Variables and Assigning Values)

A structure variable can be initialized at the moment it is declared, in a manner similar to an array — by listing the initial values for each member, in order, inside curly braces. Alternatively, an already-declared structure variable's members can be assigned individually afterward, using the **dot (.) operator**.

```
#include <stdio.h>
#include <string.h>

struct Book {
    int book_id;
    char title[50];
    float price;
};

int main() {
    // Initialization at declaration
    struct Book b1 = {101, "C Programming", 299.50};

    // Assigning values later using the dot (.) operator
    struct Book b2;
    b2.book_id = 102;
    strcpy(b2.title, "Data Structures"); // Strings require strcpy
    b2.price = 350.00;

    printf("Book 1: %s ($%.2f)\n", b1.title, b1.price);
    return 0;
}
```

Output: Book 1: C Programming (\$299.50)

Why strcpy() is needed: A structure member declared as `char title[50]` is itself an array. Arrays cannot be assigned directly using the `=` operator after declaration (only at the moment of initialization), so the standard library function `strcpy()` from `<string.h>` must be used instead to copy a string into it.

3. Memory Allocation Functions for Structures

Instead of declaring a structure variable in the ordinary way (which reserves memory for it automatically, typically on the stack, at compile time), memory for a structure can also be allocated **dynamically at runtime**, using a pointer together with a memory allocation function such as `malloc()`. This is useful when the number of structures needed is not known until the program is actually running.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Employee {
    int id;
    char name[20];
};

int main() {
    // Dynamically allocate memory for one Employee structure
    struct Employee *empPtr = (struct Employee*) malloc(sizeof(struct Employee));

    if (empPtr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // Assign data
    empPtr->id = 1;
    strcpy(empPtr->name, "Alice");

    printf("Employee ID: %d, Name: %s\n", empPtr->id, empPtr->name);

    // Always free dynamically allocated memory
    free(empPtr);
    return 0;
}
```

Output: Employee ID: 1, Name: Alice

`sizeof(struct Employee)` is used to tell `malloc()` exactly how many bytes are needed to hold one complete `Employee` structure — this is safer and clearer than hard-coding a specific byte count. As with any dynamically allocated memory, the pointer should always be checked against `NULL` before use, and `free()` must be called once the memory is no longer needed to avoid a memory leak.

4. Pointers with Structures

When a pointer refers to a structure, its members cannot be accessed using the standard dot (.) operator directly on the pointer. Instead, the **arrow operator (->)** must be used, which combines dereferencing the pointer and accessing a member into a single, convenient step.

```
#include <stdio.h>

struct Point {
    int x;
    int y;
};

int main() {
    struct Point p1 = {10, 20};
    struct Point *ptr = &p1;    // ptr holds the address of p1

    // Accessing members using the arrow operator
    printf("X coordinate: %d\n", ptr->x);
    printf("Y coordinate: %d\n", ptr->y);

    return 0;
}
```

Output: X coordinate: 10 | Y coordinate: 20

`ptr->x` is exactly equivalent to writing `(*ptr).x` — first dereferencing `ptr` to get the actual `Point` structure, then accessing its `x` member with the dot operator — but the arrow form is shorter, avoids needing extra parentheses, and is what is used almost universally in real C code.

5. Array with Structures

An array of structures is used whenever multiple records that all share the exact same structure type need to be stored together — for example, storing the data for all 60 students in a class, each represented by one 'struct Student' element in the array.

```
#include <stdio.h>

struct Student {
    int roll_no;
    float marks;
};

int main() {
    // Array to hold 3 Student records
    struct Student class[3];

    // Assigning data
    class[0].roll_no = 1; class[0].marks = 85.5;
    class[1].roll_no = 2; class[1].marks = 90.0;
    class[2].roll_no = 3; class[2].marks = 78.0;

    // Printing data using a loop
    for (int i = 0; i < 3; i++) {
        printf("Roll No: %d, Marks: %.1f\n", class[i].roll_no, class[i].marks);
    }
    return 0;
}
```

Output:

Roll No: 1, Marks: 85.5

Roll No: 2, Marks: 90.0

Roll No: 3, Marks: 78.0

Each element of the array, such as `class[0]`, is itself a complete structure, so its individual members are accessed by combining array indexing with the dot operator, as in `class[0].roll_no`. This pattern of looping over an array of structures is extremely common for processing lists of records such as students, employees, products, or transactions.

6. User Defined Function with Structures

Structures can be passed to functions in essentially the same way as ordinary variables, and the same by-value versus by-reference distinction covered in Unit 3 applies here as well:

- **By Value:** The entire structure is copied into the function's parameter. Any changes made to the parameter inside the function have no effect on the original structure in the caller.
- **By Reference (Pointer):** Only the address of the structure is passed. This avoids the overhead of copying a potentially large structure, and allows the function to directly modify the original structure's members via the arrow operator.

```
#include <stdio.h>

struct Rectangle {
    int length;
    int width;
};

// Function taking a structure BY VALUE
void printArea(struct Rectangle r) {
    printf("Area is: %d\n", r.length * r.width);
}

// Function taking a structure BY REFERENCE (Pointer)
void scaleRectangle(struct Rectangle *r, int factor) {
    r->length *= factor;    // Modifies the original struct
    r->width *= factor;
}

int main() {
    struct Rectangle box = {5, 10};

    printArea(box);          // Output: Area is: 50

    scaleRectangle(&box, 2);  // Pass address to scale by 2

    printArea(box);          // Output: Area is: 200
    return 0;
}
```

Output: Area is: 50 then Area is: 200

Choosing between the two approaches: Passing by value is simplest and safest when a function only needs to read a structure's data (as with printArea above), since the original cannot accidentally be modified. Passing by reference is preferred when the function needs to modify the caller's original structure (as with scaleRectangle), or when the structure is large and the cost of copying it would be wasteful.

7. Nested Structures

A structure can itself contain another structure as one of its members — this is called **nesting**, and it is a powerful way of building more complex, hierarchical data models out of smaller, reusable pieces. For example, an 'Employee' record might naturally contain a nested 'Date' structure to represent that employee's joining date, rather than three separate loose integer members for day, month, and year.

```
#include <stdio.h>

// Inner structure
struct Date {
    int day;
    int month;
    int year;
};

// Outer structure
struct Employee {
    int id;
    char name[20];
    struct Date join_date;    // Nested structure
};

int main() {
    struct Employee e1 = {101, "John", {15, 8, 2023}};

    // Chaining dot operators to access nested members
    printf("%s joined on %d/%d/%d\n", e1.name, e1.join_date.day, e1.join_date.month, e1.join_date.year);
    return 0;
}
```

Output: John joined on 15/8/2023

To access a member of the nested inner structure, the dot operator is simply chained: `e1.join_date.day` first reaches the 'join_date' member of `e1` (which is itself a struct `Date`), and then reaches its 'day' member. If a pointer to the outer structure were used instead, the same chaining would combine the arrow and dot operators, e.g. `ePtr->join_date.day`.

8. Introduction to Union

A **union** is a user-defined data type declared using the `union` keyword, and its syntax looks very similar to a structure's. The critical difference is in how memory is allocated: **all members of a union share the exact same block of memory**, starting at the same address, rather than each member getting its own separate space. Because every member overlaps in memory, a union can only meaningfully hold a valid value for *one* of its members at any given moment — writing to one member overwrites the underlying bytes that any other member would also be reading from.

```
#include <stdio.h>

union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    union Data data;

    data.i = 10;
    printf("data.i: %d\n", data.i);    // Works fine

    data.f = 220.5;
    // Now data.i is corrupted because data.f overwrote the shared memory
    printf("data.f: %f\n", data.f);

    return 0;
}
```

Output: data.i: 10 then data.f: 220.500000 (data.i is no longer valid at this point)

Why use a union at all? Unions are useful in situations where a variable is only ever needed to hold *one* of several possible types of data at any given time — such as a value in an interpreter that could be an int, a float, or a string depending on context, or a memory-constrained embedded system where minimizing total memory usage matters more than storing every possible value simultaneously.

9. Difference between Structure & Union

Feature	Structure (struct)	Union (union)
Memory Allocation	Allocates separate memory for each member.	Allocates one shared block of memory for all members.
Size	Size is greater than or equal to the sum of the sizes of all its members (may include padding).	Size is equal to the size of its single largest member.
Value Storage	Can store values for all of its members simultaneously.	Can store a valid value for only ONE member at a time.
Modification	Changing the value of one member does not affect the values of any other member.	Changing the value of one member corrupts (overwrites) the values of all other members.
Typical Use Case	Representing a record where every field is needed at the same time (e.g., a Student record).	Representing a value that could be one of several types, or saving memory when only one member is ever needed at once.

Illustrating the size difference: Given the union Data example above (containing an int, a float, and a 20-character array), a struct with the same three members would occupy roughly $4 + 4 + 20 = 28$ bytes (plus possible padding), since each member gets its own space. The equivalent union, by contrast, would occupy only about 20 bytes in total — the size of its largest member, the char array — since all members share that same space.

10. Enumerated Type (enum)

An **enum** (enumeration) is a user-defined data type consisting of a set of named integral constants. Its primary purpose is to assign meaningful, descriptive names to a series of integer values, making a program far more readable and self-documenting — writing Wednesday in code is much clearer at a glance than writing the bare number 3, even though both refer to exactly the same underlying value.

Default numbering: By default, the first name listed in an enum is automatically assigned the integer value 0, the second name is assigned 1, the third is assigned 2, and so on, incrementing by 1 for each subsequent name. These default values can also be overridden manually, by explicitly assigning a specific integer to any name in the list — any names that follow will then continue counting upward from that new value.

```
#include <stdio.h>

// Defining an enum for days of the week
enum Weekday {
    Sunday,      // 0
    Monday,      // 1
    Tuesday,     // 2
    Wednesday,   // 3
    Thursday,    // 4
    Friday,      // 5
    Saturday     // 6
};

int main() {
    enum Weekday today;
    today = Wednesday;    // today holds the integer value 3

    if (today == Wednesday) {
        printf("It is the middle of the week! (Value: %d)\n", today);
    }

    return 0;
}
```

Output: It is the middle of the week! (Value: 3)

Example — manually overriding default values:

```
enum StatusCode {
    OK = 200,
    NOT_FOUND = 404,
    SERVER_ERROR = 500
};
// Here, none of the values follow the automatic 0,1,2... sequence,
// since every name has been explicitly assigned its own value.
```

Why use enum instead of #define or plain integers? Unlike a series of #define macros, an enum groups all of its related named constants together under one single named type, which many compilers can use for additional type-checking (e.g., warning if a variable of one enum type is compared against a completely unrelated enum's value). This makes enums a cleaner, safer, and more maintainable way of working with a fixed set of related named options — such as days of the week, directions, states in a state machine, or status/error codes.